

Codex AI Development Transcript

COMS3011A Lab 1 Todo Application | Prepared 2026-08-02

This transcript records the main Codex-assisted development decisions, corrections and verification activities for the local-first todo application. It focuses on substantive engineering work and omits routine terminal output and minor operational details.

Project Architecture and Structure Decisions

The application was kept as a single Next.js project with a lightweight layered architecture inside one process. The React UI calls Next.js Route Handlers, Route Handlers translate HTTP concerns, the service layer owns business rules, and the repository layer owns SQLite persistence.

Application shell	app/page.js renders the active task dashboard; app/archived/page.js renders the archived view using the same TaskManager component with an archived flag.
UI components	components/ contains the header, task manager, forms, cards, list, modal, archive dialog, statistics and scoped CSS module.
Service rules	lib/taskService.js coordinates validation, not-found handling, update/archive decisions and derived overdue information.
Persistence	lib/taskRepository.js maps JavaScript task objects to SQL rows and exposes parameterized CRUD-style task operations.
Database setup	lib/database.js initializes the schema from database/schema.sql and shares one connection through globalThis for development reload stability.

Important decision: Overdue was intentionally modeled as derived information instead of a persisted status. This kept the database status constraint small and avoided stale overdue data when the date changes.

Database Implementation

SQLite with better-sqlite3 was selected because the project is local-first, single-user and needs persistence across restarts without external infrastructure. The schema stores active and archived tasks in one tasks table because tasks are the only persisted resource.

Schema constraints	Required title, description, due_date and topic fields use NOT NULL and trimmed-content checks where appropriate. Status is constrained to Todo, In-Progress or Complete.
Archiving model	Archive is represented by archived_at. Active tasks have archived_at as NULL; archived tasks retain their row and receive an archive timestamp.

Sorting model	Repository sorting uses fixed SQL expressions for due date, case-insensitive topic and logical status order, with id as a deterministic secondary key.
Safety	User-provided values are passed through prepared statements. Sort expressions come only from an internal allowlist, not direct request input.
Initialization	createDatabase() creates the data directory for file-backed databases, enables foreign keys, enables WAL for the local file database and runs the idempotent schema.

REST API Implementation

The REST API is implemented with Node.js Next.js Route Handlers under `app/api/tasks`. Responses consistently use a `data` property for success and a structured `error` property for validation, missing-task and unexpected failures.

GET /api/tasks	Lists active or archived tasks with <code>archived=true</code> or <code>archived=false</code> and validates <code>sortBy</code> as <code>dueDate</code> , <code>status</code> or <code>topic</code> .
POST /api/tasks	Creates a task from title, description, <code>dueDate</code> , topic and optional status; successful creation returns 201 and a Location header.
GET /api/tasks/[id]	Validates the ID through the service and returns 404 when the task is absent.
PATCH /api/tasks/[id]	Updates editable fields, or archives when the request body is exactly <code>{ archived: true }</code> . No delete endpoint was added.
Error handling	<code>TaskValidationError</code> maps to 400, <code>TaskNotFoundError</code> maps to 404, and unexpected errors are logged server-side and returned as 500.

UI Integration

The UI was integrated around `TaskManager` as the main client component. It coordinates loading, sorting, modal state, save/archive flows, validation messages, success feedback and archived-count refreshes.

Data loading	<code>TaskManager</code> fetches active or archived task lists with cache disabled and uses <code>AbortController</code> to avoid setting state after a cancelled request.
Create and edit	<code>TaskModal</code> and <code>TaskForm</code> send POST or PATCH requests, display field-level API validation errors and refresh the visible task list after success.
Archive flow	<code>ArchiveDialog</code> confirms the action, sends PATCH with <code>archived: true</code> , reloads active tasks and updates the archived count.

Task display	TaskCard shows topic, due date, status and the derived overdue badge supplied by the service layer.
Accessibility	Forms and dialogs include labels, aria-invalid/error wiring, role alert messaging, keyboard-friendly controls and a skip link.

Testing Approach and Verification

Verification combined deterministic automated tests with linting and a production build. Repository and service tests inject in-memory SQLite databases, so they verify persistence behavior without relying on the generated local data/tasks.db file.

Repository tests	Create, retrieve, edit, archive and logical status sorting were tested against isolated in-memory databases.
Rules tests	Valid statuses, impossible calendar dates, overdue derivation and validation-error output were tested separately from storage.
Service tests	Derived overdue data, validation rejection and missing-task errors were tested through the service contract.
Latest checks	ESLint passed, the Next.js production build completed successfully, and Vitest reported 3 test files passed with 10 tests passed.

Important Debugging Issue: Service and Repository Export Mismatch

Problem: During service integration, the service layer and repository module did not expose/consume the same contract. The service expected repository operations for create, find by id, list, update and archive, while the repository exports initially did not line up with those expected names and call signatures. This created an integration failure at the service boundary rather than inside SQL itself.

Investigation: The issue was traced through the request path from the Route Handlers into createTaskService() and then into the defaultRepository object. Reviewing lib/taskService.js and lib/taskRepository.js showed that the API routes depended on service methods, the service depended on a repository interface, and the actual repository exports had to match that interface exactly.

Correction: The fix was to align the repository public API and the service imports around the final named functions: createTask, findTaskById, findTasks, updateTask and archiveTask. The service now builds defaultRepository directly from those named imports, and tests inject the same contract with an in-memory database to prevent future drift.

Verification: After the correction, the service tests exercised create, update, overdue derivation and not-found behavior through the same repository contract. The API routes also import only service-level operations, keeping HTTP code decoupled from repository internals.

Development Corrections and Outcomes

Status modeling	Unsupported statuses were rejected and the status set was narrowed to Todo, In-Progress and Complete. Overdue remained derived rather than stored.
Archive behavior	Deletion was avoided. Archiving became a reversible-history-friendly timestamp update with a separate archived view.
Date validation	The database checks YYYY-MM-DD shape while taskRules rejects impossible calendar dates before persistence.
API consistency	Successful responses and error responses were standardized so UI code could reliably display data or field-level validation messages.
UI robustness	Loading, empty, retry, saving, validation and success states were wired into the client flow instead of leaving silent failures.

Final result: the project has a concise layered structure, local SQLite persistence, a small REST API, an integrated React interface, deterministic tests and a documented resolution for the major service/repository contract mismatch.

Document Scope

This transcript summarizes meaningful development work only.